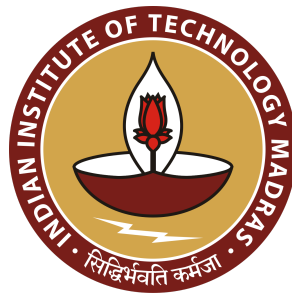


GRAND CHALLENGE - 2025

Verilog IP Porting with Shakti SoC



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

Introduction

This document details porting of custom verilog IP porting with Shakti.

Steps:

Go to Soc.defines and do the following.

1. Update the mixed cluster address range if required (fig. 1).

```
`define UARTClusterBase      'h0001_1300
`define UARTClusterEnd      'h0001_1540
`define SPIClusterBase      'h0002_0000
`define SPIClusterEnd      'h0002_01FF
`define PWMClusterBase      'h0003_0000
`define PWMClusterEnd      'h0003_05FF
`define MixedClusterBase    'h0004_0000
`define MixedClusterEnd    'h0004_15FF
`define EthBase             'h0004_4000
`define EthEnd              'h0004_7FFF
// - PWM cluster
`define PWMCluster_Num_Slaves 2
`define PWMCluster_Num_Masters 1
```

Fig. 1

2. Add a new slave number and memory range for the newly added IP (Fig. 2).

```
// - - Mixed Cluster
`define MixedCluster_Num_Slaves 7
`define MixedCluster_Num_Masters 1

`define I2C0_slave_num 0
`define I2C1_slave_num 1
`define PLIC_slave_num 2
`define GPIO_slave_num 3
`define XADC_slave_num 4
`define Pinmux_slave_num 5
`define MixedCluster_err_slave_num 6

`define I2C0Base . . . 'h0004_0000
`define I2C0End . . . . 'h0004_00FF
`define GPIOBase → . . 'h0004_0100
`define GPIOEnd → . . 'h0004_01FF
`define XADCBase . . . . 'h0004_1000
`define XADCEnd . . . . 'h0004_13FF
`define I2C1Base . . . . 'h0004_1400
`define I2C1End . . . . 'h0004_14FF
`define PinmuxBase . . . . . 'h0004_1500
`define PinmuxConfigReg 'h0004_1510
`define PinmuxEnd . . . . . 'h0004_15FF
`define PLICBase → . . 'h0C00_0000
`define PLICEnd → . . . . 'h0FFF_FFFF
```

Fig. 2

E.g. add the following in the fig. 2

A.

```
// - Mixed Cluster  
`define MixedCluster_Num_Slaves 8  
`define MixedCluster_Num_Masters 1
```

B.

```
`define Pinmux_slave_num 5  
`define I2S_slave_num  
`define MixedCluster_err_slave_num 7
```

C.

```
`define GPIOBase 'h0004_0100  
`define GPIOEnd 'h0004_01FF  
`define I2S_Base... 'h0004_0400  
`define I2S_End.... 'h0004_04FF  
`define XADCBBase 'h0004_1000  
`define XADCEnd 'h0004_13FF
```

Go to mixed_cluster.bsv and do the following.

- Add else if case for the new device (Fig.3)

```

function Bit#(TLog#(`MixedCluster_Num_Slaves)).fn_slave_map.(Bit#
(`paddr).addr);
....Bit#(TLog#(`MixedCluster_Num_Slaves)).slave_num.=.0;
....if(addr>=`GPIOBase.&&addr<=`GPIOEnd)
.....slave_num.=.`GPIO_slave_num;
....else if(addr>=`PLICBase.&&addr<=`PLICEnd)
.....slave_num.=.`PLIC_slave_num;
....else if(addr>=`I2C1Base.&&addr<=`I2C1End)
.....slave_num.=.`I2C1_slave_num;
....else if(addr>=`I2C0Base.&&addr<=`I2C0End)
.....slave_num.=.`I2C0_slave_num;
....else if(addr>=`XADCBase.&&addr<=`XADCEnd)
.....slave_num.=.`XADC_slave_num;
....else if(addr>=`PinmuxBase.&&addr<=`PinmuxEnd)
.....slave_num.=.`Pinmux_slave_num;
....else
.....slave_num.=.`MixedCluster_err_slave_num;
....
....return slave_num;
endfunction:fn_slave_map

```

Fig. 3

For e.g. add the following.

```

if(addr>=`GPIOBase && addr<=`GPIOEnd)
| slave_num = `GPIO_slave_num;
....if(addr>=`I2S_Base.&&addr<=`I2S_END)
....| slave_num.=.`I2S_slave_num;
else if(addr>=`PLICBase && addr<=`PLICEnd)
| slave_num = `PLIC_slave_num;

```

- Add interface for the new device (fig.4 & fig. 5). This is needed when we want to make external AXI4 connectivity.

```

interface Ifc_mixed_cluster;
    method I2C_out i2c0_out;           //I2c IO interface
    method I2C_out i2c1_out;           //I2c IO interface
    method Bit#(2) sb_ext_interrupt;
    (*always_ready, always_enabled*)
    interface GPIO#(32) gpio_io;       //GPIO IO interface
    interface IOCellSide pinmuxtop_iocell_side;
    interface PeripheralSide pinmuxtop_peripheral_side;
    (*always_ready, always_enabled*)
    method Action interrupts(Bit#(13) inp);
    interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
    ... interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) xadc_master;
endinterface

```

Fig. 4

Add the following AXI interface for I2S.

```

interface Ifc_mixed_cluster;
    method I2C_out i2c0_out;           //I2c IO interface
    method I2C_out i2c1_out;           //I2c IO interface
    method Bit#(2) sb_ext_interrupt;
    (*always_ready, always_enabled*)
    interface GPIO#(32) gpio_io;       //GPIO IO interface
    interface IOCellSide pinmuxtop_iocell_side;
    interface PeripheralSide pinmuxtop_peripheral_side;
    (*always_ready, always_enabled*)
    method Action interrupts(Bit#(13) inp);
    interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
    interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) xadc_master;
    ... interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) i2s_master;
endinterface

```

Make I2S AXI interface connectivity with core AXI.

I.e. Add the following

```

method I2C_out i2c0_out= i2c0.io;
method I2C_out i2c1_out= i2c1.io;
method sb_ext_interrupt = wr_sb_ext_interrupt;
interface gpio_io= gpio.io;
method Action interrupts(Bit#(13) inp);
| wr_external_interrupts<= inp;
endmethod
interface pinmux_top_iocell_side = pinmux_top.pinmux_axi4lite_iocell_side;
interface pinmux_top_peripheral_side = pinmux_top.pinmux_axi4lite_peripheral_side;
interface slave= c2s_xactor.axi_side;
interface xadc_master = fabric.v_to_slaves[`XADC_slave_num];
... interface i2s_master = fabric.v_to_slaves[`I2S_slave_num];

```

To

```

method I2C_out i2c0_out= i2c0.io;
method I2C_out i2c1_out= i2c1.io;
method sb_ext_interrupt = wr_sb_ext_interrupt;
interface gpio_io= gpio.io;
method Action interrupts(Bit#(13) inp);
| wr_external_interrupts<= inp;
endmethod
interface pinmux_top_iocell_side = pinmux_top.pinmux_axi4lite_iocell_side;
interface pinmux_top_peripheral_side = pinmux_top.pinmux_axi4lite_peripheral_side;
interface slave= c2s_xactor.axi_side;
... interface xadc_master = fabric.v_to_slaves[`XADC_slave_num];

```

Go to fpga_top.v and do the following.

Add the required IO ports.

```

output [3:0] phy_ex_data,
output phy_ref_clk,
inout phy_mdio,
output phy_mdc,
// ---- JTAG ports ---- //
// ---- UART ports ---- //
input      uart0_SIN,
output     uart0_SOUT,
... output i2s_sd,
... output i2s_sck,
... output i2s_ws,
...

```

Declare wire for the new device's AXI4 signals.

```

.. // XADC Axi4-Lite Slave
.. wire xadc_master_awvalid;
.. wire [11-1:0] xadc_master_awaddr;
.. wire xadc_master_m_awready_awready;
.. wire xadc_master_wvalid;
.. wire [32-1:0] xadc_master_wdata;
.. wire [(32/8)-1:0] xadc_master_wstrb;
.. wire xadc_master_m_wready_wready;
.. wire xadc_master_m_bvalid_bvalid;
.. wire [1:0] xadc_master_m_bvalid_bresp;
.. wire xadc_master_bready;
.. wire xadc_master_arvalid;
.. wire [11-1:0] xadc_master_araddr;
.. wire xadc_master_m_arready_arready;
.. wire xadc_master_m_rvalid_rvalid;
.. wire [1:0] xadc_master_m_rvalid_rresp;
.. wire [32-1:0] xadc_master_m_rvalid_rdata;
.. wire xadc_master_rready;

```

Similarly add the following for I2S interface.


```

...// I2S-Axi4-Lite-Slave
...wire i2s_master_awvalid;
...wire [11-1:0] i2s_master_awaddr;
...wire i2s_master_m_awready_awready;
...wire i2s_master_wvalid;
...wire [32-1:0] i2s_master_wdata;
...wire [(32/8)-1:0] i2s_master_wstrb;
...wire i2s_master_m_wready_wready;
...wire i2s_master_m_bvalid_bvalid;
...wire [1:0] i2s_master_m_bvalid_bresp;
...wire i2s_master_bready;
...wire i2s_master_arvalid;
...wire [11-1:0] i2s_master_araddr;
...wire i2s_master_m_arready_arready;
...wire i2s_master_m_rvalid_rvalid;
...wire [1:0] i2s_master_m_rvalid_rresp;
...wire [32-1:0] i2s_master_m_rvalid_rdata;
...wire i2s_master_rready;

```

Make IP connectivity in mkdebugSoc or mkSoC.

```

... // XADC connection
... .xadc_master_awvalid(xadc_master_awvalid),
... .xadc_master_awaddr(xadc_master_awaddr),
... .xadc_master_m_awready_awready(xadc_master_m_awready_awready),
... .xadc_master_wvalid(xadc_master_wvalid),
... .xadc_master_wdata(xadc_master_wdata),
... .xadc_master_wstrb(xadc_master_wstrb),
... .xadc_master_m_wready_wready(xadc_master_m_wready_wready),
... .xadc_master_m_bvalid_bvalid(xadc_master_m_bvalid_bvalid),
... .xadc_master_m_bvalid_bresp(xadc_master_m_bvalid_bresp),
... .xadc_master_bready(xadc_master_bready),
... .xadc_master_arvalid(xadc_master_arvalid),
... .xadc_master_araddr(xadc_master_araddr),
... .xadc_master_m_arready_arready(xadc_master_m_arready_arready),
... .xadc_master_m_rvalid_rvalid(xadc_master_m_rvalid_rvalid),
... .xadc_master_m_rvalid_rresp(xadc_master_m_rvalid_rresp),
... .xadc_master_m_rvalid_rdata(xadc_master_m_rvalid_rdata),
... .xadc_master_rready(xadc_master_rready),

```

Similarly add the following.

```

// XADC connection
.xadc_master_awvalid(xadc_master_awvalid),
.xadc_master_awaddr(xadc_master_awaddr),
.xadc_master_m_awready_awready(xadc_master_m_awready_awready),
.xadc_master_wvalid(xadc_master_wvalid),
.xadc_master_wdata(xadc_master_wdata),
.xadc_master_wstrb(xadc_master_wstrb),
.xadc_master_m_wready_wready(xadc_master_m_wready_wready),
.xadc_master_m_bvalid_bvalid(xadc_master_m_bvalid_bvalid),
.xadc_master_m_bvalid_bresp(xadc_master_m_bvalid_bresp),
.xadc_master_bready(xadc_master_bready),
.xadc_master_arvalid(xadc_master_arvalid),
.xadc_master_araddr(xadc_master_araddr),
.xadc_master_m_arready_arready(xadc_master_m_arready_arready),
.xadc_master_m_rvalid_rvalid(xadc_master_m_rvalid_rvalid),
.xadc_master_m_rvalid_rresp(xadc_master_m_rvalid_rresp),
.xadc_master_m_rvalid_rdata(xadc_master_m_rvalid_rdata),
.xadc_master_rready(xadc_master_rready),

.....// I2S connection
......i2s_master_awvalid(i2s_master_awvalid),
......i2s_master_awaddr(i2s_master_awaddr),
......i2s_master_m_awready_awready(i2s_master_m_awready_awready),
......i2s_master_wvalid(i2s_master_wvalid),
......i2s_master_wdata(i2s_master_wdata),
......i2s_master_wstrb(i2s_master_wstrb),
......i2s_master_m_wready_wready(i2s_master_m_wready_wready),
......i2s_master_m_bvalid_bvalid(i2s_master_m_bvalid_bvalid),
......i2s_master_m_bvalid_bresp(i2s_master_m_bvalid_bresp),
......i2s_master_bready(i2s_master_bready),
......i2s_master_arvalid(i2s_master_arvalid),
......i2s_master_araddr(i2s_master_araddr),
......i2s_master_m_arready_arready(i2s_master_m_arready_arready),
......i2s_master_m_rvalid_rvalid(i2s_master_m_rvalid_rvalid),
......i2s_master_m_rvalid_rresp(i2s_master_m_rvalid_rresp),
......i2s_master_m_rvalid_rdata(i2s_master_m_rvalid_rdata),
......i2s_master_rready(i2s_master_rready),

```

Make instance of new device:

```

xadc_wiz_0.xadc_wiz_inst.(
.....s_axi_aclk.....(core_clk),
.....s_axi_aresetn...(~soc_reset),
.....s_axi_awaddr....(xadc_master_awaddr),
.....s_axi_awvalid... (xadc_master_awvalid),
.....s_axi_awready... (xadc_master_m_awready_awready),
.....s_axi_wdata.....(xadc_master_wdata),
.....s_axi_wstrb.....(xadc_master_wstrb),
.....s_axi_wvalid....(xadc_master_wvalid),
.....s_axi_wready... (xadc_master_m_wready_wready),
.....s_axi_bresp.....(xadc_master_m_bvalid_bresp),
.....s_axi_bvalid....(xadc_master_m_bvalid_bvalid),
.....s_axi_bready... (xadc_master_bready),
.....s_axi_araddr....(xadc_master_araddr),
.....s_axi_arvalid... (xadc_master_arvalid),
.....s_axi_arready... (xadc_master_m_arready_arready),
.....s_axi_rdata.....(xadc_master_m_rvalid_rdata),
.....s_axi_rresp.....(xadc_master_m_rvalid_rresp),
.....s_axi_rvalid....(xadc_master_m_rvalid_rvalid),
.....s_axi_rready... (xadc_master_rready),
.....vauxp0.(vauxp0),
.....vauxn0.(vauxn0),
.....vauxp4.(vauxp4),
.....vauxn4.(vauxn4),
.....vauxp5.(vauxp5),
.....vauxn5.(vauxn5),
.....vauxp6.(vauxp6),
.....vauxn6.(vauxn6),
.....vauxp7.(vauxp7),
.....vauxn7.(vauxn7),
.....vauxp12.(vauxp12),
.....vauxn12.(vauxn12),
.....vauxp13.(vauxp13),
.....vauxn13.(vauxn13),
.....vauxp14.(vauxp14),
.....vauxn14.(vauxn14),
.....vauxp15.(vauxp15),
.....vauxn15.(vauxn15),
.....vp_in.(vp_in),
.....vn_in.(vn_in),
.....temp_out(device_temp),.....//output.wire[11::0].temp_out
.....ip2intc_irpt(xadc_ip2intc_irpt)
.....);

```

Similarly add your interface.

Make an instance of interrupt if required.

In mkSoC instance, add

```
.ext_interrupts_i(interrupts)
```

```
axi_ethernetlite_0 eth10 (  
  .s_axi_aclk(core_clk),          // input wire s_axi_aclk  
  .s_axi_aresetn(~soc_reset),     // input wire s_axi_aresetn  
  .ip2intc_irpt(eth10_ip2intc_irpt), // output wire ip2intc_irpt  
  .s_axi_awaddr    (eth_master_awaddr),  
  .s_axi_awvalid   (eth_master_awvalid),  
  .s_axi_awready   (eth_master_m_awready_awready),  
  .s_axi_wdata     (eth_master_wdata),  
  .s_axi_wstrb     (eth_master_wstrb),  
  .s_axi_wvalid    (eth_master_wvalid),  
  .s_axi_wready    (eth_master_m_wready_wready),  
  .s_axi_bresp     (eth_master_m_bvalid_bresp),  
  .s_axi_bvalid    (eth_master_m_bvalid_bvalid),  
  .s_axi_bready    (eth_master_bready),  
  .s_axi_araddr    (eth_master_araddr),  
  .s_axi_arvalid   (eth_master_arvalid),  
  .s_axi_arready   (eth_master_m_arready_arready),  
  .s_axi_rdata     (eth_master_m_rvalid_rdata),  
  .s_axi_rresp     (eth_master_m_rvalid_rresp),  
  .s_axi_rvalid    (eth_master_m_rvalid_rvalid),  
  .s_axi_rready    (eth_master_rready),  
  .phy_tx_clk(phy_tx_clk),        // input wire phy_tx_clk  
  .phy_rx_clk(phy_rx_clk),        // input wire phy_rx_clk  
  .phy_crs(phy_crs),              // input wire phy_crs  
  .phy_dv(phy_dv),               // input wire phy_dv  
  .phy_rx_data(phy_rx_data),      // input wire [3 : 0] phy_rx_data  
  .phy_col(phy_col),              // input wire phy_col  
  .phy_rx_er(phy_rx_er),          // input wire phy_rx_er  
  .phy_rst_n(phy_rst_n),          // output wire phy_rst_n  
  .phy_tx_en(phy_tx_en),          // output wire phy_tx_en  
  .phy_tx_data(phy_tx_data),      // output wire [3 : 0] phy_tx_data  
  .phy_mdio_i(phy_mdio_i),        // input wire phy_mdio_i  
  .phy_mdio_o(phy_mdio_o),        // output wire phy_mdio_o  
  .phy_mdio_t(phy_mdio_t),        // output wire phy_mdio_t  
  .phy_mdc(phy_mdc)               // output wire phy_mdc  
);
```

Add IOBUF if bidirectional pins are there.

```

IOBUF gpio_iobuf_inst (
    .O(gpio_in[index]),
    .IO(gpio[index]),
    .I(gpio_out[index]),
    .T(~gpio_out_en[index])
);
end
endgenerate*/

//-----IOBUF.FOR.I2C-----//
...IOBUF.eth_mdio_inst(
.....O(phy_mdio_i),
.....IO(phy_mdio),
.....I(phy_mdio_o),
.....T(phy_mdio_t)
.....);

//IOBUF i2c0_scl_inst(
//      .O(i2c0_scl_in),

```

Update the constraints file:

```

## ChipKit I2C
#set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports { i2c1_scl }]; #IO_L4P_T0_D04_14
Sch=ck_scl
#set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports { i2c1_sda }]; #IO_L4N_T0_D05_14
Sch=ck_sda

set_property -dict { PACKAGE_PIN C2 IOSTANDARD LVCMOS33 } [get_ports { sys_rst }]; #IO_L16P_T2_35 Sch=ck_rst

# SMSC Ethernet PHY
set_property -dict { PACKAGE_PIN D17 IOSTANDARD LVCMOS33 } [get_ports { phy_col }]; #IO_L16N_T2_A27_15
Sch=eth_col
set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { phy_crs }]; #IO_L15N_T2_DQS_ADV_B_15
Sch=eth_crs
set_property -dict { PACKAGE_PIN F16 IOSTANDARD LVCMOS33 } [get_ports { phy_mdc }]; #IO_L14N_T2_SRCC_15
Sch=eth_mdc
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports { phy_mdio }]; #IO_L17P_T2_A26_15
Sch=eth_mdio
set_property -dict { PACKAGE_PIN G18 IOSTANDARD LVCMOS33 } [get_ports { phy_ref_clk }]; #IO_L22P_T3_A17_15
Sch=eth_ref_clk
set_property -dict { PACKAGE_PIN C16 IOSTANDARD LVCMOS33 } [get_ports { phy_rst_n }]; #IO_L20P_T3_A20_15
Sch=eth_rstn
set_property -dict { PACKAGE_PIN F15 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_clk }]; #IO_L14P_T2_SRCC_15
Sch=eth_rx_clk
set_property -dict { PACKAGE_PIN G16 IOSTANDARD LVCMOS33 } [get_ports { phy_dv }]; #IO_L13N_T2_MRCC_15
Sch=eth_rx_dv
set_property -dict { PACKAGE_PIN D18 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_data[0] }];
#IO_L21N_T3_DQS_A18_15 Sch=eth_rxd[0]
set_property -dict { PACKAGE_PIN E17 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_data[1] }]; #IO_L16P_T2_A28_15
Sch=eth_rxd[1]
set_property -dict { PACKAGE_PIN E18 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_data[2] }]; #IO_L21P_T3_DQS_15
Sch=eth_rxd[2]
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_data[3] }]; #IO_L18N_T2_A23_15
Sch=eth_rxd[3]
set_property -dict { PACKAGE_PIN C17 IOSTANDARD LVCMOS33 } [get_ports { phy_rx_er }]; #IO_L20N_T3_A19_15
Sch=eth_rxerr
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_clk }]; #IO_L13P_T2_MRCC_15
Sch=eth_tx_clk
set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_en }]; #IO_L19N_T3_A21_VREF_15
Sch=eth_tx_en
set_property -dict { PACKAGE_PIN H14 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_data[0] }]; #IO_L15P_T2_DQS_15
Sch=eth_txd[0]
set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_data[1] }]; #IO_L19P_T3_A22_15
Sch=eth_txd[1]
set_property -dict { PACKAGE_PIN J13 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_data[2] }]; #IO_L17N_T2_A25_15
Sch=eth_txd[2]
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { phy_tx_data[3] }]; #IO_L18P_T2_A24_15
Sch=eth_txd[3]

```

Now we have 2 more cases.

1. Verilog files with AXI4/AXI4 lite interface.

Following the above said procedure is sufficient.

2. Verilog files with WB interface or any other interface.

Use one of the following files and add WB to AXI bridge and use it.

<https://opencores.org/projects/ax4lbr>

<https://github.com/germit/WishboneAXI/tree/master/cores/Wishbone2AXI/hdl>

<https://github.com/wallento/wb2axi/blob/master/src/wb2axi.sv>

<https://github.com/ZipCPU/wb2axip/blob/master/rtl/axlite2wbsp.v>

Note: It depends on the project interconnections and few will vary based on the code.